Webber — iOS App Design Specification

Nov 5, 2026 · @Matthew Sinclair

Webber is an iOS app that renders folders of static web files as if a local server served them, with no server. This document is the build spec for Claude Code (sections 1–15), the feature tiers, ads and Pro unlock (16–19), the Webble protocol and creator-to-consumer distribution (20–21), and the open questions to validate on device (22). Screen mockups live on the Webber iOS Mockups canvas.

1. Purpose and scope

Webber is an iOS app that renders folders of static HTML, CSS and JavaScript from the user's cloud storage inside a web view, as if a local web server served them, without running a server. The user opens a page by tapping a Home Screen shortcut or any `webber://` link.

Core use case. The user keeps small self-contained web pages (no backend) in iCloud Drive or Dropbox. They edit the files on a Mac. They want each page one tap away on the iPhone Home Screen, with relative links, stylesheets, scripts and `fetch()` of local files all working.

Target platform. iOS 17.0 and later, iPhone and iPad, Swift 5.9+, SwiftUI, WebKit, App Intents. Xcode 15 or later. No third-party dependencies.

In scope

- Register one or more site folders (iCloud container by default; any Files-accessible folder by picker)
- Serve registered folders through a `WKURLSchemeHandler` with correct MIME types
- Open pages via the `webber://` custom URL scheme
- Expose an `OpenPage` App Intent so Shortcuts can place a page on the Home Screen
- Keep a local cache of each site so pages open offline
- A minimal management UI: list sites, add, remove, resync, open

Out of scope (non-goals)

- No HTTP server, no sockets, no `localhost`
- No server-side logic: no PHP, no templating, no databases
- No Dropbox SDK or OAuth; Dropbox is reached only through the Files app's File Provider
- No service workers, no push notifications, no web app manifest handling
- No editing of files inside the app

- No placing of icons on the Home Screen by the app itself (iOS does not permit it; Shortcuts does this)
- No Mac Catalyst or macOS target in v1

Tiers. Sections 16 to 19 split the feature set into a Free tier (one site, one page, interstitial ads) and a one-time Pro unlock. Sections 1 to 15 describe the full engine; the gates in section 18.3 decide what each tier can reach.

2. Glossary

Term	Meaning
Site	A folder of static web files registered with Webber. One site = one folder.
Site alias	A short identifier for a site, used as the host part of a `webber://` URL. Lowercase ASCII letters, digits and hyphens, 1–32 characters, must start with a letter. Example: `notes`.
Site root	The folder the alias points at. Either a subfolder of the app's iCloud container or an external folder held by a security-scoped bookmark.
Launch URL	A `webber://` URL that opens the app and selects a page. Example: `webber://notes/todo/index.html`.
Internal URL	A `webber-local://` URL the web view loads. The scheme handler serves it from the cache. Never leaves the app. Example: `webber-local://notes/todo/index.html`.
Cache	A per-site copy of the site root under the app's Caches directory. The handler serves only from here.
Entry page	The page a site opens when the launch URL gives no path. Defaults to `index.html`.

3. User-facing behaviour

First run. The app creates its iCloud container and shows an empty site list with one instruction: put a folder in the Webber folder in Files, or tap Add to pick a folder. The Webber iCloud folder appears in Files under iCloud Drive within a few seconds of first launch.

Adding a site from the iCloud container. The app watches the container root. Each top-level subfolder becomes a site automatically, with its alias derived from the folder name (lowercased, non-alphanumerics replaced by hyphens, trimmed). The user does not need to register these.

Adding an external site. The user taps Add, picks a folder in the document picker (Dropbox, On My iPhone, any File Provider), and gives it an alias. The app stores a security-scoped bookmark and syncs the folder into the cache.

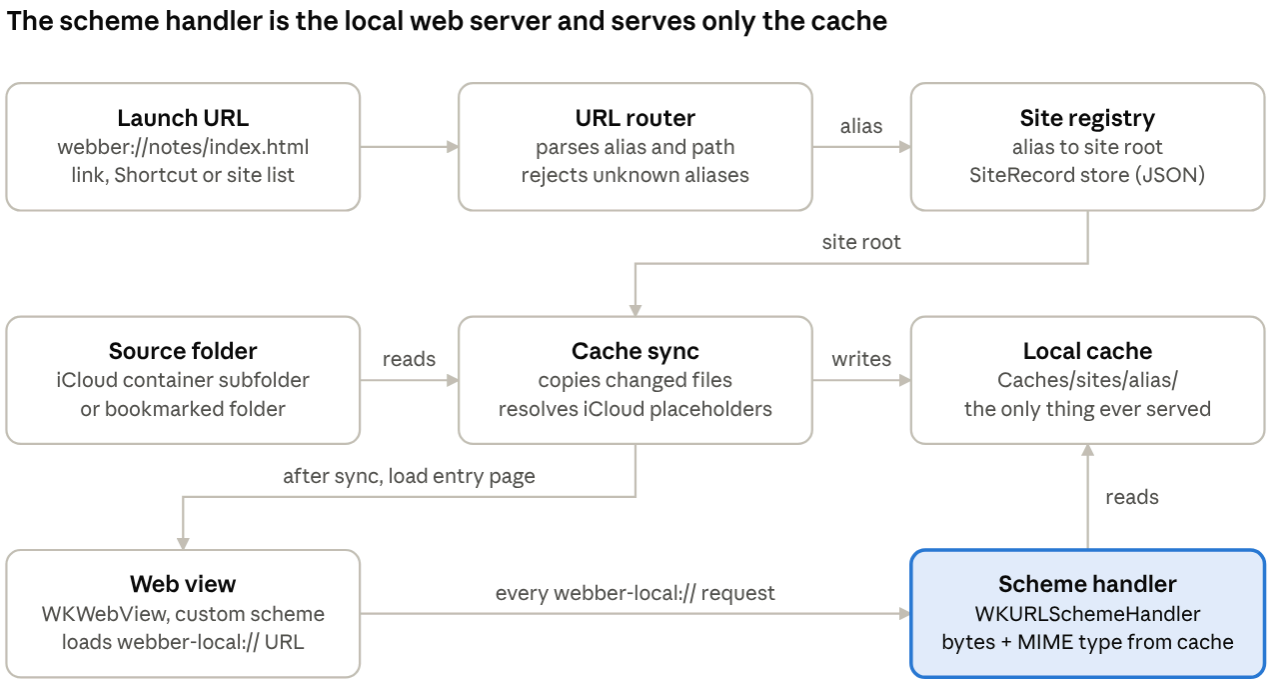
Opening a page. The user taps a `webber://` link, a Home Screen shortcut, or a site in the list. The app syncs the site if it is stale, then shows the page full screen in a web view with a small toolbar: back, forward, reload, resync, close.

Home Screen flow. The user opens Shortcuts, creates a shortcut with the Webber action Open Page, picks the site and page, then uses Add to Home Screen with a chosen icon. Webber shows a one-line explanation of this in the site detail screen and a button that opens Shortcuts.

Offline. Pages open from the cache. If the cache is empty and the source is unreachable, the app shows an error page with the site name and a Retry button.

Editing on the Mac. The user edits files in the folder on the Mac. The next time they open the page, Webber detects the change and resyncs before rendering. Reload in the toolbar forces a resync.

4. Architecture overview



Webber data flow · launch URL to rendered page

A launch URL names a site and a path. The router resolves the alias through the registry, the sync layer brings the site root into the local cache, and the web view then loads the page through the internal scheme. The handler never touches the source folder; it reads only the cache, so pages render offline and the source never serves half-synced files.

Modules (one Swift file group each)

Module	Responsibility	Depends on
`WebberApp`	App entry, `onOpenURL`, scene setup	Router
`Router`	Parses `webber://`, resolves site, drives navigation state	SiteRegistry, CacheSync
`SiteRegistry`	Persists `SiteRecord` values, derives container sites, resolves bookmarks	Foundation
`CacheSync`	Copies a site root into the cache, detects change, handles iCloud download	SiteRegistry
`SchemeHandler`	`WKURLSchemeHandler` serving `webber-local://` from the cache	SiteRegistry
`PageView`	`UIViewRepresentable` wrapping `WKWebView`, toolbar	SchemeHandler

Module	Responsibility	Depends on
`Intents`	`OpenPageIntent`, `SiteEntity`, `AppShortcutsProvider`	Router
`UI`	Site list, site detail, add-site sheet, settings	All

5. URL schemes

Two schemes, with different jobs. `webber://` is public and opens the app. `webber-local://` is private and is registered only on the app's `WKWebViewConfiguration`.

5.1 Launch scheme: `webber://`

Grammar:

```
webber://<alias>[/<path>][?<query>][#<fragment>]
```

- `alias` is the host component. It must match `^[a-z][a-z0-9-]{0,31}$`. Compare case-insensitively; lowercase before lookup.
- `path` is optional. Empty or `/` means the site's entry page. A path ending in `/` resolves to `<path>/index.html`.
- `query` and `fragment` are passed through unchanged to the internal URL so page JavaScript can read them.
- Register `webber` in `CFBundleURLTypes` with role Viewer.

Examples:

Launch URL	Resolves to internal URL
`webber://notes`	`webber-local://notes/index.html`
`webber://notes/`	`webber-local://notes/index.html`
`webber://notes/todo/`	`webber-local://notes/todo/index.html`
`webber://notes/todo/list.html?day=mon#top`	`webber-local://notes/todo/list.html?day=mon#top`
`webber://Notes/A.html`	`webber-local://notes/A.html` (alias lowercased, path case preserved)

Rejections, each shown as an in-app error (never a crash):

- Alias fails the regex, or no site has that alias

- Path contains `..`, a NUL byte, or percent-encoded equivalents after decoding
- Path, after decoding, escapes the site root (check with `standardizedFileURL` and prefix comparison)
- Path points at a directory with no `index.html`

5.2 Internal scheme: `webber-local://`

Same grammar. The handler treats `<alias>` as the cache folder name and `<path>` as a relative file path inside it. Every page gets origin `webber-local://<alias>`, so relative links, `fetch('./data.json')`, `<script type="module">` and `localStorage` work and are scoped per site.

5.3 Mapping rules in `URLParser`

```
struct PageRequest: Equatable {
  let alias: String        // lowercased, validated
  let relativePath: String // no leading slash, index.html appended where
needed
  let query: String?
  let fragment: String?

  var internalURL: URL     // webber-local://alias/relativePath?
query#fragment
}

enum URLParseError: Error { case badScheme, badAlias, badPath,
pathEscapesRoot }

func parseLaunchURL(_ url: URL) -> Result<PageRequest, URLParseError>
```

The same parser handles both schemes; only the scheme check differs.

6. Site registry and storage

6.1 `SiteRecord` model

```
struct SiteRecord: Codable, Identifiable, Equatable {
  enum Source: Codable, Equatable {
    case container(folderName: String)       // subfolder of the iCloud
container
    case external(bookmark: Data)             // security-scoped bookmark
  }
  let id: UUID
  var alias: String
  var displayName: String
  var source: Source
  var entryPage: String            // default "index.html"
  var lastSyncedAt: Date?
  var lastSyncFingerprint: String? // see section 7
}
```

Persist the array as JSON at `Application Support/sites.json`. Write atomically. Container-derived records are regenerated on every launch and on container change; external records are only added or removed by the user.

6.2 iCloud container

- Enable the iCloud capability with iCloud Documents and one container, e.g. `iCloud.com.geodica.webber`.
- Set `NSUbiquitousContainers` in Info.plist so the folder is visible in Files: `NSUbiquitousContainerIsDocumentScopePublic` = true, `NSUbiquitousContainerName` = Webber, `NSUbiquitousContainerSupportedFolderLevels` = Any.
- Site roots are `<container>/Documents/<folderName>/`. Call `FileManager.default.url(forUbiquityContainerIdentifier:)` on a background queue; it blocks and returns nil if iCloud is off.
- Watch `<container>/Documents` with `NSMetadataQuery` scoped to `NSMetadataQueryUbiquitousDocumentsScope` to refresh the site list when folders appear or disappear.
- Alias derivation: folder name lowercased, every run of non-`[a-z0-9]` replaced by one hyphen, hyphens trimmed from both ends, truncated to 32, prefixed with `s-` if it does not start with a letter. On collision, append `-2`, `-3`.

6.3 External folders

- Present `UIDocumentPickerViewController(forOpeningContentTypes: [.folder])` with `shouldShowFileExtensions` off.

- On selection: `startAccessingSecurityScopedResource()`, create a bookmark with `.minimalBookmark` options (no `.withSecurityScope` on iOS), store it, then `stopAccessingSecurityScopedResource()`.
- Resolve with `URL(resolvingBookmarkData:options:[],relativeTo:nil,bookmarkDataIsStale:)`. If stale, re-save the refreshed bookmark. If resolution fails, mark the site as needing re-authorisation and show that state in the list.
- Wrap every access in `startAccessing` / `stopAccessing` with `defer`. Never hold access open outside a sync.

6.4 Why not serve the source folder directly

External folders can only be read while access is open, iCloud files may be placeholders, and both can change mid-page-load. The cache removes all three problems and makes the handler trivial.

7. Cache sync

The cache lives at `Library/Caches/sites/<alias>/`. It mirrors the site root exactly. `CacheSync` is an `actor` so two syncs of one site never overlap.

7.1 Sync algorithm (`sync(site:force:)`)

1. Resolve the site root (container path or bookmark). Open security-scoped access if external.
2. Enumerate the root recursively with `FileManager.enumerator(at:includingPropertiesForKeys:)`, requesting `.isDirectoryKey`, `.contentModificationDateKey`, `.fileSizeKey`, `.ubiquitousItemDownloadingStatusKey`. Skip hidden files, `.DS_Store`, and any entry whose path depth exceeds 32.
3. For each file whose iCloud downloading status is not `.current`, call `startDownloadingUbiquitousItem(at:)`. Poll with `NSMetadataQuery` or re-read the resource key every 250 ms, up to a per-site timeout of 20 s. On timeout, continue with what downloaded and report `partial`.
4. Compute a fingerprint: SHA-256 over the sorted list of `relativePath|size|modificationDate` strings. If it equals `lastSyncFingerprint` and `force` is false, stop; the cache is current.
5. Build the new tree in `Library/Caches/sites/<alias>.tmp/`: create directories, copy each file with `FileManager.copyItem`.
6. Atomically swap: `replaceItemAt(_:withItemAt:)` on the site cache directory. Delete the old tree.

7. Store the fingerprint and `lastSyncedAt` in the registry. Close security-scoped access.

7.2 When sync runs

Trigger	Behaviour
Opening a page	Fingerprint check only (step 1–4). Copy only if changed. Show the cached page immediately if the check takes longer than 300 ms, then reload once the sync completes if it changed anything.
Reload button	`force: true`, full copy.
App becomes active	Fingerprint check for the currently shown site only.
Site added	Full sync, with a progress indicator.
Site removed	Delete `Library/Caches/sites/<alias>/`.

7.3 Offline

If the root is unreachable (bookmark resolution fails, container nil, enumeration throws), sync returns `.unavailable` and the router serves the existing cache without complaint. Only an empty cache produces a user-facing error.

7.4 Limits

- Files over 50 MB are skipped and logged. This is a static-page app, not a media server.
- A site over 500 MB total aborts the sync with an error shown in the site list.
- iOS may purge `Caches` under storage pressure. Sync restores it on next open; this is the reason the cache holds nothing authoritative.

8. Rendering

8.1 `WKWebView` configuration

```
let config = WKWebViewConfiguration()
config.setURLSchemeHandler(SchemeHandler(cacheRoot: cacheRoot), forURLScheme:
"webber-local")
config.defaultWebpagePreferences.allowsContentJavaScript = true
config.allowsInlineMediaPlayback = true
config.mediaTypesRequiringUserActionForPlayback = []
config.websiteDataStore = .default()   // persists localStorage per site
origin
```

One `WKWebView` per `PageView`. Do not share the web view between sites; origin isolation comes from the alias host, but a fresh view avoids leaking navigation history.

Set `webView.isInspectable = true` in debug builds so Safari Web Inspector on the Mac can attach.

8.2 Navigation policy (`WKNavigationDelegate`)

Requested URL	Action
`webber-local://<same alias>/…`	Allow
`webber-local://<other alias>/…`	Allow; the handler serves it (cross-site links between the user's own sites are a feature)
`webber://…`	Cancel; hand to the router, which navigates in the same view
`http://`, `https://`, `mailto:`, `tel:`, `sms:`	Cancel; open with `UIApplication.shared.open`
`file://`, `about:`, anything else	Cancel

8.3 `SchemeHandler` contract

```
final class SchemeHandler: NSObject, WKURLSchemeHandler {
  init(cacheRoot: URL)
  func webView(_:start task: WKURLSchemeTask)
  func webView(_:stop task: WKURLSchemeTask)
}
```

Behaviour of `start`:

1. Parse `task.request.url` with the shared parser. On failure respond 400.
2. Resolve to `cacheRoot/<alias>/<relativePath>`. Standardise. If the result does not have `cacheRoot/<alias>/` as a prefix, respond 403.
3. If the path is a directory, append `index.html`. If no file exists, respond 404 with a small HTML body naming the missing path.
4. Read the file with `Data(contentsOf:options: .mappedIfSafe)`.
5. Build an `HTTPURLResponse` with status 200 and headers: `Content-Type` from the MIME table, `Content-Length`, `Cache-Control: no-cache`, `Access-Control-Allow-Origin: *`.
6. Call `task.didReceive(response)`, `task.didReceive(data)`, `task.didFinish()`.

7. Track in-flight tasks in a `Set<ObjectIdentifier>` guarded by a lock. `stop` removes the task; before every `task.didReceive` / `didFinish` call check the task is still in the set, because calling into a stopped task crashes.

Do the file read on a background queue and deliver on the main queue. Support `Range` requests for audio and video: parse `bytes=a-b`, respond 206 with `Content-Range`. Ignore it for other types.

8.4 MIME table

Use `UTType(filenameExtension:)?.preferredMIMEType` first, then this override table, then `application/octet-stream`:

Extension	MIME type
html, htm	text/html; charset=utf-8
css	text/css; charset=utf-8
js, mjs	text/javascript; charset=utf-8
json, map	application/json; charset=utf-8
svg	image/svg+xml
wasm	application/wasm
woff2	font/woff2
webmanifest	application/manifest+json
md, txt	text/plain; charset=utf-8

ES modules fail without the `text/javascript` type, and `.mjs` is not in `UTType` on all iOS versions, so the override is required.

8.5 Known limitations to document in the README

- Service workers do not register on custom schemes.
- `navigator.share`, geolocation and camera may refuse because the origin is not `https`. Test each API the pages need.
- `window.open` is ignored unless a `WKUIDelegate` is implemented; v1 implements `createWebViewWith` to load the URL in the same view and return nil.
- Form `POST` has no server. The handler responds 405 to any method other than GET and HEAD.

9. App Intents and Shortcuts

The intent is what makes the Home Screen story usable. Without it the user has to type a `webber://` URL into an Open URL action by hand.

9.1 Entities

```
struct SiteEntity: AppEntity {
  static var typeDisplayRepresentation: TypeDisplayRepresentation = "Site"
  static var defaultQuery = SiteQuery()
  var id: String            // alias
  var displayName: String
  var displayRepresentation: DisplayRepresentation { .init(title: "\
(displayName)") }
}

struct SiteQuery: EntityQuery {
  func entities(for identifiers: [String]) async throws -> [SiteEntity]
  func suggestedEntities() async throws -> [SiteEntity]   // all registered
sites
}
```

Pages are not entities. Enumerating every HTML file in every site is slow and noisy. The page is a free-text parameter with a sensible default.

9.2 `OpenPageIntent`

```
struct OpenPageIntent: AppIntent {
  static var title: LocalizedStringResource = "Open Page"
  static var description = IntentDescription("Opens a page from one of your
Webber sites.")
  static var openAppWhenRun = true

  @Parameter(title: "Site") var site: SiteEntity
  @Parameter(title: "Page", default: "index.html") var page: String

  static var parameterSummary: some ParameterSummary {
    Summary("Open \(\.$page) in \(\.$site)")
  }

  @MainActor
  func perform() async throws -> some IntentResult {
    try Router.shared.open(alias: site.id, path: page)
    return .result()
  }
}
```

- `openAppWhenRun = true` brings Webber to the foreground; the router then shows the page.
- Validation errors (unknown alias, bad path) throw an `IntentError` with a readable message; Shortcuts shows it.
- Optionally provide a `PageOptionsProvider` that lists the top-level `*.html` files in the chosen site's cache, so the Page field offers suggestions without being an entity.

9.3 `AppShortcutsProvider`

Register one App Shortcut for `OpenPageIntent` with phrases such as "Open (.applicationName) page". This surfaces the action in Spotlight and Siri without the user building anything.

9.4 Getting a page onto the Home Screen

iOS has no API that lets an app place an icon on the Home Screen, and App Review does not change that. One tap in Webber gets the user to the edge; the last taps happen in Apple's apps. Three routes ship, in this order of priority, and all three are Free.

Route	Taps after leaving Webber	How it works	Limits
Pre-built shortcut (primary)	4: Add Shortcut › ⓘ › Add to Home Screen › Add	Webber opens `shortcuts://import-shortcut?url=<signed .shortcut>&name=<site>`. Webber Cloud generates and signs one file per webble link; the import sheet shows a finished Open Page shortcut.	Signed files need `shortcuts sign` on macOS, so generation is server-side. Local folder sites get `shortcuts://create-shortcut` plus the three written steps instead. Verify the import scheme on iOS 17 and 18 (section 22.5).
Widget (parallel)	6: long-press › + › Webber › Add Widget › tap › pick site	Small widget = one site, medium = up to four, Lock Screen = one. Each cell is a `Link` to `webber://<alias>`.	A widget tile, not an icon. No Shortcuts involved; every tap is familiar.
Safari web clip (fallback for webbles)	4: Share › Add to Home Screen › Add	The `/w/<token>` landing page carries the webble's icon and title; Safari adds it.	Each open bounces Safari → Webber. Only for devices without App Clips or users who refuse Shortcuts.

Also available with no icon: the Action button on iPhone 15 Pro and later bound to the Open Page intent, Lock Screen widgets, and Spotlight.

In the app. The site detail screen has one primary button, Add to Home Screen, which runs the shortcut route, and a secondary link, Add a widget instead, which shows the six steps. The three written Shortcuts steps appear only when the pre-built file is unavailable. The app never claims to have placed an icon; it says what the next tap is.

9.5 Widgets (milestone 9, Free)

A `WidgetKit` widget listing up to four sites, each row a `Link` to `webber://<alias>`. This gives a Home Screen entry point with no Shortcuts step at all, at the cost of a widget-sized footprint rather than an icon.

10. UI specification

SwiftUI throughout. `NavigationStack` with a path bound to router state so deep links and intents push the page view directly.

Mockups of every screen below are on the Webber iOS Mockups canvas: site list, page view, site detail, add site, webble install card, paywall, interstitial and settings. The canvas is the visual reference; this table is the behavioural one.

Screen	Contents	Actions
Site list (root)	One row per site: display name, alias in secondary text, status badge (Synced, Syncing, Needs access, Error), last synced relative time. Empty state explains the iCloud folder and the Add button.	Tap row → page view. Swipe → Resync, Remove (external only). Toolbar: Add, Settings.
Add site sheet	Folder picker button, alias field pre-filled from folder name and validated live, entry page field (default `index.html`).	Save, Cancel.
Site detail	Display name, alias, source path, entry page (editable), cache size, last sync, fingerprint prefix. Home Screen instructions and Open Shortcuts button.	Open, Resync, Clear cache, Remove.
Page view	Full-screen `WKWebView`. Thin top bar with site name and Close. Bottom bar: Back, Forward, Reload (force sync), Share current `webber://` URL. Bars hide on scroll down, show on scroll up.	Standard navigation.
Error page	Rendered inside the web view as HTML served by the handler from a bundled template. Shows site, requested path, reason, Retry.	Retry re-runs sync then reloads.
Settings	Toggle: resync on open (default on). Toggle: keep screen awake in page view. Clear all caches. Version and build.	—

Status mapping. `SiteState` enum: `.synced(Date)`, `.syncing(progress: Double?)`, `.needsAccess`, `.error(String)`, `.neverSynced`. The list observes an `@Observable` `SiteStore` that publishes `[SiteRecord]` plus `[alias: SiteState]`.

Deep link while a page is open. The router replaces the current page view rather than pushing a second one. One page view on the stack at any time.

iPad. Same screens, `NavigationSplitView` with the site list in the sidebar and the page view in detail.

11. Error handling and edge cases

Situation	Behaviour
iCloud signed out or iCloud Drive off	Container URL is nil. Container-derived sites vanish from the list. Show a banner: "iCloud Drive is off. Turn it on in Settings to use the Webber folder." External sites keep working.
Two folders derive the same alias	Second gets `-2`. Show both aliases in the list so the user can rename the folder.
Folder renamed on the Mac	Alias changes; old shortcuts break. Keep the old cache for 7 days under the old alias and show "Moved" in the list with the new alias, so the user can fix shortcuts.
`webber://` link received while the app is cold	`onOpenURL` fires after the scene exists; the router stores the request and the root view consumes it once `SiteStore` has loaded. Never drop a launch URL.
Launch URL for an unknown alias	Show the site list with an alert naming the alias. Do not crash, do not open a blank page.
Page references `../shared/style.css` that escapes the site	Handler returns 403. Document that sites must be self-contained.
Symlinks in the site root	Skip during sync. `enumerator` with `.skipsHiddenFiles`; check `isSymbolicLink` resource key and skip.
File changes while a sync copies it	The tmp-dir-then-replace swap ensures the served cache is always a complete snapshot. A half-copied file in tmp is discarded.

Situation	Behaviour
Security-scoped bookmark stops resolving	State becomes `.needsAccess`. Row shows a Reauthorise button that opens the picker; the new bookmark replaces the old one and the alias is kept.
Low disk	`copyItem` throws. Abort sync, keep the previous cache, state `.error`.
Very large site	Sync aborts over 500 MB (section 7.4). Error names the size.
Page JavaScript calls `fetch('https://…')`	Allowed; the web view does ordinary network requests. Add `NSAppTransportSecurity` exceptions only if the user asks.
Background launch by Shortcuts automation	`openAppWhenRun = true` means the intent never runs headless.
Multiple scenes (iPad)	Each scene has its own router; the shared `SiteStore` and `CacheSync` actor are process-wide singletons.

12. Project structure, build targets and configuration

```
Webber/
  Webber.xcodeproj
  Webber/
    App/
      WebberApp.swift          // @main, onOpenURL, scene
      Router.swift             // launch URL → navigation state
    Model/
      SiteRecord.swift
      SiteState.swift
      PageRequest.swift
    Registry/
      SiteStore.swift          // @Observable, persistence
      ContainerWatcher.swift   // NSMetadataQuery on the iCloud container
      AliasDerivation.swift
      BookmarkStore.swift
    Sync/
      CacheSync.swift          // actor
      Fingerprint.swift
      UbiquityDownloader.swift
```

```
    Web/
      SchemeHandler.swift
      MIMETypes.swift
      URLParser.swift
      PageView.swift           // UIViewRepresentable
      PageCoordinator.swift    // WKNavigationDelegate, WKUIDelegate
      ErrorPage.html           // bundled template
    Intents/
      SiteEntity.swift
      OpenPageIntent.swift
      WebberShortcuts.swift
    UI/
      SiteListView.swift
      SiteDetailView.swift
      AddSiteSheet.swift
      SettingsView.swift
    Resources/
      Assets.xcassets
      Info.plist
      Webber.entitlements
  WebberTests/
    URLParserTests.swift
    AliasDerivationTests.swift
    SchemeHandlerTests.swift
    CacheSyncTests.swift
    FingerprintTests.swift
  WebberUITests/
  README.md
```

Targets. `Webber` (iOS app), `WebberTests` (unit), `WebberUITests` (UI). No Swift Package dependencies. Deployment target iOS 17.0.

Info.plist keys

Key	Value
`CFBundleURLTypes`	One entry: `CFBundleURLSchemes` = `["webber"]`, `CFBundleURLName` = bundle id, role Viewer
`NSUbiquitousContainers`	As in section 6.2
`UIFileSharingEnabled`	false
`LSSupportsOpeningDocumentsInPlace`	false (sites are copied, not opened in place)
`UISupportsDocumentBrowser`	false

Entitlements

Key	Value
`com.apple.developer.icloud-container-identifiers`	`["iCloud.<bundle id>"]`
`com.apple.developer.icloud-services`	`["CloudDocuments"]`
`com.apple.developer.ubiquity-container-identifiers`	same container

Code style. Swift 5.9, strict concurrency warnings on, `@MainActor` on all UI types, `actor` for `CacheSync`, no force unwraps outside tests, errors as typed enums conforming to `LocalizedError`. Two-space indentation.

13. Testing

13.1 Unit tests (XCTest)

Suite	Cases
`URLParserTests`	Every row of the examples table in 5.1; each rejection case; alias case folding; query and fragment pass-through; percent-encoded `..`; Unicode in path preserved.
`AliasDerivationTests`	`My Notes` → `my-notes`; `2024 Plans` → `s-2024-plans`; `--a--` → `a`; 40-character name truncated to 32; collision suffixing.
`SchemeHandlerTests`	Use a temp cache root and a fake `WKURLSchemeTask` (protocol-wrap the task). Assert 200 + MIME for html/css/js/mjs/json/svg/woff2; 404 for missing; 403 for traversal; 405 for POST; 206 for a `Range` request on a binary; `stop` before completion produces no further callbacks.
`CacheSyncTests`	Temp source and cache dirs. First sync copies all; unchanged source is a no-op (fingerprint equal); modified file is re-copied; deleted file disappears from cache; nested dirs; hidden files skipped; swap is atomic (old cache intact if copy throws midway, simulated by a read-only file).

Suite	Cases
`FingerprintTests`	Deterministic across runs; order-independent; changes on size, date or path.

13.2 UI tests

- Launch with `-UITestSeed` argument that creates a fixture site in a temp container; assert the list shows it and tapping opens a page whose `<title>` the test reads via the accessibility tree.
- Deep link test: launch with `webber://fixture/sub/` and assert the page view appears with the expected title.

13.3 Manual test plan (on device, before each release)

- [ ] Put a folder in iCloud Drive/Webber on a Mac; it appears in the list on the phone within 30 s
- [ ] Open it; relative CSS, JS modules and `fetch('./data.json')` all work
- [ ] Edit a file on the Mac, reopen on the phone; change is visible
- [ ] Enable Airplane Mode; page still opens from cache
- [ ] Pick a Dropbox folder via Files; it syncs and opens
- [ ] Kill the app, reboot the phone, open the Dropbox site; bookmark still resolves
- [ ] Build a Shortcut with Open Page, add to Home Screen, tap the icon; page opens in under 2 s
- [ ] Tap a `webber://` link in Notes; app opens on the right page
- [ ] Tap an `https://` link inside a page; Safari opens
- [ ] `localStorage` written by site A is not visible from site B
- [ ] Rename the folder on the Mac; list shows Moved with the new alias

14. Implementation plan for Claude Code

Build in this order. Each milestone ends with its tests green and a commit. Do not start the next until the previous runs on a device or simulator.

1. **Scaffold.** Xcode project, targets, folder layout from section 12, Info.plist URL type, entitlements, empty `SiteListView`. Build runs.
2. **URL parsing and alias derivation.** `URLParser`, `PageRequest`, `AliasDerivation`, reserved aliases. All `URLParserTests` and `AliasDerivationTests` pass. No UI.
3. **Scheme handler with a bundled fixture.** `SchemeHandler`, `MIMETypes`, `PageView`, `PageCoordinator`. Ship a fixture site inside the bundle, copy it to the cache on launch, and open `webber-local://fixture/index.html` from a debug button. Verify

CSS, module JS and `fetch` of a local JSON file work. `SchemeHandlerTests` pass. Run the section 22 item 1 secure-context checks here.

4. **Registry and iCloud container.** `SiteRecord`, `SiteStore`, `ContainerWatcher`, `webber.json` parsing (apply nothing Pro yet). Folders dropped into the iCloud Webber folder appear in the list.
5. **Cache sync.** `CacheSync`, `Fingerprint`, `UbiquityDownloader`. Opening a container site syncs and renders it. `CacheSyncTests` and `FingerprintTests` pass. Offline open works.
6. **Launch scheme and router.** `Router`, `onOpenURL`, cold-start handling, navigation policy, error page template, share link and QR.
7. **Entitlements and gates.** `EntitlementStore`, `FeatureGate`, `RemoteConfig` with bundled fallback, paywall sheet, `-ForcePro`. Free site limit and second-HTML-document gate enforced. StoreKit configuration file for local testing.
8. **Ads.** `AdProvider`, `HouseAdProvider`, `AdScheduler`, `AdView`, inventory fetch and cache, local measurement, debug menu. Every policy rule in 19.3 has a unit test with a fake clock.
9. **App Intents.** `SiteEntity`, `OpenPageIntent`, `WebberShortcuts`. Pre-built shortcut import and the widget target (Home and Lock Screen) ship here; the Home Screen flow in 9.4 works end to end in both tiers.
10. **External folders.** `BookmarkStore`, `AddSiteSheet`, reauthorise flow, Pro gate. A Dropbox folder picked via Files syncs and opens after a reboot.
11. **Pro display and manifest features.** Theme colour, status bar, orientation, bars, keep awake, site lock, live reload.
12. **Bridge.** `window.webber` with stub on Free; bridge tests through a fixture page that exercises every method.
13. **Import paths and webbles.** Zip import, templates, then the Webble protocol: manifest parsing, install card, remote sync with hash verification, universal links, `BGAppRefreshTask` update checks, `make-webble.sh`.
14. **Surfaces.** Control Center control (iOS 18), Action button binding, Spotlight indexing.
15. **Sync and polish.** Registry sync via KVS, config export/import, dev tools, iPad split view, README with the limitations from 8.5. UI tests pass. App Store privacy label drafted.

Milestones 2, 3, 5 and 8 are the risk. Ship to TestFlight after milestone 9; everything after it is additive. If the scheme handler cannot satisfy a page the user cares about (section 22, item 1), stop and reassess before milestone 4.

16. Feature matrix and tiers

Two tiers. Free gives one site, its entry page, Home Screen shortcuts and ads. Webber Pro is a one-time unlock that removes ads and opens everything else. Every gate goes through one `FeatureGate` type (section 18.3) so the split can move without touching feature code.

Design decisions behind the split

- The Open Page intent stays free. It is the only usable Home Screen path, and a free tier that cannot reach the Home Screen is a demo, not a product.
- "One page" means one HTML document: the entry page and its assets. A single-page app with hash routing fits inside the free tier by design; that is acceptable, since it still needs Pro for a second site.
- Ads exist to convert, not to earn. The default inventory is house ads for Pro. Third-party inventory is pluggable (section 19) but off by default.
- Limits are remote-configurable (section 19.6) so the free site count or page cap can change without a release.

#	Feature	Tier	Notes
1	One site from the iCloud Webber folder	Free	First folder by name wins; others show as locked in the list
2	Scheme handler, full MIME table, module JS, `fetch` of local files	Free	Section 8
3	Local cache and offline open	Free	Section 7
4	`webber://` launch links	Free	Section 5
5	Open Page App Intent, Shortcuts, Siri, Spotlight action	Free	Section 9
6	Share `webber://` link as text or QR code	Free	QR generated with `CIFilter.qrCodeGenerator`
7	Reload and pull-to-refresh (forced sync)	Free	Section 7.2
8	Full-page interstitial ads	Free	Free only; Pro never instantiates the ad provider
9	Unlimited sites	Pro	Limit read from remote config, default 1 free

#	Feature	Tier	Notes
10	Multi-page navigation inside a site	Pro	Free: a second `.html` navigation shows the paywall
11	External folders via Files (Dropbox, On My iPhone, any provider)	Pro	Section 6.3
12	Site manifest `webber.json` (name, icon, theme colour, orientation, bars)	Pro	Section 17.1
13	Live reload: page reloads when the source folder changes	Pro	`NSMetadataQuery` on container sites; poll every 5 s for external while the page is open
14	Zip import: drop `site.zip` in the Webber folder, or share a zip to the app	Pro	Extracted with `Compression` / `Archive` framework into a folder named after the zip
15	Webbles: subscribe to a remote site by manifest URL (section 20)	Free	Free to install up to `freeWebbleLimit`; update checks via `BGAppRefreshTask`
16	Widgets: small and medium Home Screen, Lock Screen, Control Center control (iOS 18)	Free	Free: a Home Screen path for webble consumers (section 9.4). Each entry is a `Link` to `webber://alias`
17	Spotlight indexing of pages	Pro	`CSSearchableItem` per `.html` with its `<title>`; tap opens via launch URL
18	JavaScript bridge `window.webber`	Pro	Section 17.2
19	Face ID or passcode lock per site	Pro	`LAContext`; flag in the site record, not the manifest
20	Developer tools: in-app console, JS error overlay, Web Inspector toggle	Pro	Console via `WKScriptMessageHandler` injected `console.*` shim
21	Display options: keep awake, orientation lock, hide bars, status bar style	Pro	Manifest sets defaults; site detail overrides
22	Registry sync across devices	Pro	`NSUbiquitousKeyValueStore` for container sites and settings; external bookmarks never sync (device-bound)

#	Feature	Tier	Notes
23	Site templates: generate a starter site into the Webber folder	Pro	Three bundled: blank, dashboard, notes
24	Export and import app configuration	Pro	JSON file via share sheet; excludes bookmarks

Paywall trigger points. Adding a second site, picking an external folder, navigating to a second HTML document, toggling any Pro setting, and the Remove Ads button on every interstitial. Each trigger passes a `reason` so the paywall can lead with the feature the user just tried.

17. Site manifest and JavaScript bridge

17.1 `webber.json`

An optional file at the site root. Read during sync, validated, stored on the `SiteRecord` as `manifest: SiteManifest?`. Every key is optional; unknown keys are ignored.

```
{
  "name": "Daily Notes",
  "alias": "notes",
  "entry": "index.html",
  "icon": "icon.png",
  "themeColor": "#1b1b1f",
  "statusBar": "light",
  "orientation": "any",
  "bars": "auto",
  "keepAwake": false,
  "liveReload": true,
  "version": "1.2.0"
}
```

Key	Type	Effect
`name`	string ≤ 40	Display name in the list and page bar
`alias`	string, section 5.1 regex	Overrides the derived alias. Collision rules still apply. Free tier ignores it.
`entry`	relative path	Entry page; default `index.html`

Key	Type	Effect
`icon`	relative path to PNG ⩾ 180 px	Shown in the list, the widget, Spotlight and the Shortcuts picker. Copied into the cache at sync.
`themeColor`	`#rrggbb`	Page bar tint and launch background
`statusBar`	`light`, `dark`, `auto`	Status bar style while the page is shown
`orientation`	`any`, `portrait`, `landscape`	Pro only; ignored on Free
`bars`	`auto`, `hidden`, `shown`	Toolbar behaviour; Pro only
`keepAwake`	bool	`isIdleTimerDisabled` while shown; Pro only
`liveReload`	bool	Pro only
`version`	string	Shown in site detail; triggers nothing

The manifest never grants anything the tier does not. The gate is checked at apply time, not at parse time.

17.2 `window.webber` bridge

Injected by a `WKUserScript` at document start, only for `webber-local://` documents, only on Pro. Implemented with `WKScriptMessageHandlerWithReply` so every call returns a Promise. Free tier injects a stub whose methods reject with `WebberError: pro-required`, so pages can feature-detect with `webber.tier`.

```
interface Webber {
  readonly version: string;           // app version
  readonly tier: 'free' | 'pro';
  readonly site: { alias: string; name: string };
  open(alias: string, path?: string): Promise<void>;   // navigate to another
site
  share(text: string, url?: string): Promise<void>;    // system share sheet
  haptic(kind: 'light' | 'medium' | 'heavy' | 'success' | 'warning' |
'error'): Promise<void>;
  copy(text: string): Promise<void>;
  openExternal(url: string): Promise<void>;            // https only
  storage: {                                           // per-site KV, synced
via iCloud KVS, 64 KB cap
    get(key: string): Promise<string | null>;
    set(key: string, value: string): Promise<void>;
    remove(key: string): Promise<void>;
    keys(): Promise<string[]>;
```

```
  };
  files: {                                              // read-only view of
the site cache
    list(dir?: string): Promise<{ path: string; size: number; modified:
string }[]>;
    read(path: string): Promise<string>;                // text files ≤ 4 MB
  };
  resync(): Promise<{ changed: boolean }>;
  setBars(mode: 'auto' | 'hidden' | 'shown'): Promise<void>;
}
```

Security rules for the bridge

- Message handlers are registered on the `WKUserContentController` with names prefixed `webber.`; the handler checks `message.frameInfo.securityOrigin.protocol == "webber-local"` and rejects anything else.
- `openExternal` accepts `https` only and opens `SFSafariViewController`.
- `files.read` goes through the same path-escape check as the scheme handler.
- `storage` keys are namespaced `site.<alias>.<key>` in `NSUbiquitousKeyValueStore`; total per site capped at 64 KB because KVS has a 1 MB store limit.
- No bridge method can write to the site folder. Sites are read-only from the device.

17.3 Reserved aliases

`_settings`, `_sites`, `_error`, `_ad` and anything starting with `_` are reserved. `webber://_sites` opens the site list; `webber://_settings` opens settings. A folder whose derived alias would start with `_` gets the `s-` prefix.

18. Monetisation: Webber Pro

One non-consumable in-app purchase through StoreKit 2. No subscription in v1; a utility this small does not justify renewal friction, and a one-time price is the honest match for a one-time unlock.

18.1 Product

Item	Value
Product ID	`com.geodica.webber.pro`
Type	Non-consumable

Item	Value
Family Sharing	On
Price	Set in App Store Connect; not hard-coded anywhere
Restore	Restore Purchases button on the paywall and in Settings; calls `AppStore.sync()`

18.2 `EntitlementStore`

```
@MainActor @Observable
final class EntitlementStore {
  private(set) var isPro: Bool
  private(set) var lastVerified: Date?

  func start()        // reads Transaction.currentEntitlements, then listens
to Transaction.updates
  func purchase() async throws -> PurchaseOutcome   // .success, .pending,
.cancelled
  func restore() async throws
}
```

- On launch, read the cached `isPro` from `UserDefaults` immediately so the UI does not flash the free state, then verify against `Transaction.currentEntitlements` and overwrite.
- Verify `Transaction.verified` only; a `.unverified` result is treated as not Pro and logged.
- Offline: the cached value stands. There is no expiry on a non-consumable, so no grace-period logic is needed.
- Debug builds read a `-ForcePro YES|NO` launch argument so every screen can be tested in both tiers.

18.3 `FeatureGate`

The single source of truth for what a tier allows. Feature code asks the gate; it never reads `isPro` directly.

```
enum Feature: String, CaseIterable {
  case multipleSites, multiPage, externalFolders, manifest, liveReload,
zipImport,
       fetchFromURL, widgets, spotlight, bridge, siteLock, devTools,
displayOptions,
       registrySync, templates, configExport, noAds
}

struct FeatureGate {
  let entitlements: EntitlementStore
  let config: RemoteConfig          // section 19.6

  func allows(_ feature: Feature) -> Bool
  func freeSiteLimit() -> Int       // config.freeSiteLimit, default 1
  func check(_ feature: Feature, reason: PaywallReason) throws   // throws
.proRequired(reason)
}
```

`RemoteConfig.freeFeatures: [String]` can promote any feature to free without a release. It cannot demote; the bundled default is the floor.

18.4 Paywall view

- Sheet, not full-screen cover, so the user can swipe it away.
- Leads with the feature named in `PaywallReason`, then the full Pro list from section 16, then one Buy button showing the localised price from `Product.displayPrice`, then Restore, then links to Terms and Privacy.
- Never shown more than once per app session unless the user triggers it again.
- On `.pending` (Ask to Buy) show a message and keep the free tier until the transaction arrives through `Transaction.updates`.

18.5 App Review notes

- The free tier must be a complete, usable app: one site, one page, Home Screen shortcut, no time limit. It is.
- Restore Purchases must be reachable without purchasing. It is in Settings.
- Interstitials must be dismissable and must not imitate system UI. Section 19.4 covers this.
- No tracking means no App Tracking Transparency prompt and a short privacy nutrition label: Identifiers none, Usage data none unless the beacon in 19.5 is enabled, in which case declare it.

19. Interstitial ads (Free tier)

One full-page interstitial per page per day by default, with delivery policy and inventory read from a remote JSON file the app owner controls. The ad layer is a protocol so a third-party SDK can replace the built-in provider later without touching the router.

19.1 Protocol

```
protocol AdProvider: AnyObject {
  func prepare() async                                    // fetch inventory,
warm creatives
  func nextAd(for context: AdContext) async -> AdCreative?
  func record(_ event: AdEvent, creative: AdCreative)   // impression, click,
dismiss, skip
}

struct AdContext { let alias: String; let path: String; let localDay: String;
let appOpenCount: Int }

struct AdCreative: Identifiable, Codable {
  let id: String
  let kind: Kind                 // .html(URL) | .image(URL) | .bundled(name)
  let clickURL: URL?             // https, or webber://_paywall
  let skipAfterSeconds: Int
  let weight: Int
}

enum AdEvent { case impression, click, dismiss, skip }
```

`HouseAdProvider` is the v1 implementation. `AdMobProvider` is a named placeholder with no code; adding it is a separate decision because it brings tracking, a privacy label change and an SDK dependency.

19.2 Inventory file

Fetched from `RemoteConfig.adInventoryURL` (default: a URL on the app owner's domain; a bundled `ads.json` is the fallback and ships with two house creatives for Pro). Cached with ETag; refreshed at most every 6 hours and on `prepare()`. A failed fetch keeps the last good copy.

```
{
  "version": 3,
  "enabled": true,
  "policy": {
    "frequencyCap": { "count": 1, "per": "page", "window": "day" },
    "minIntervalSeconds": 120,
```

```
    "graceAppOpens": 3,
    "graceHoursAfterInstall": 24,
    "skipAfterSeconds": 5,
    "maxPerDayTotal": 6,
    "excludePaths": ["_error", "_ad"]
  },
  "creatives": [
    {
      "id": "pro-2026-11",
      "kind": "html",
      "url": "https://example.com/ads/pro.html",
      "clickURL": "webber://_paywall?reason=ad",
      "weight": 70,
      "skipAfterSeconds": 4,
      "start": "2026-11-01",
      "end": "2026-12-31",
      "maxImpressionsPerDay": 3,
      "minAppVersion": "1.0",
      "locales": ["en"],
      "enabled": true
    }
  ]
}
```

Policy key	Meaning	Default
`frequencyCap.count` / `per` / `window`	Impressions allowed per key per window. `per` is `page` (alias + path), `site` (alias) or `app`. `window` is `day`, `hour` or `session`.	1 / page / day
`minIntervalSeconds`	No ad within this many seconds of the last dismissal, across all pages	120
`graceAppOpens`	No ads during the first N app launches	3
`graceHoursAfterInstall`	No ads for this long after first launch	24
`skipAfterSeconds`	Default seconds before Skip appears; a creative may override	5
`maxPerDayTotal`	Hard ceiling across all pages	6
`excludePaths`	Alias or path prefixes that never show ads	reserved aliases

Creative selection: filter by `enabled`, date range, `minAppVersion`, locale, per-creative daily count; then weighted random by `weight`. Zero eligible creatives means no ad, never a blank interstitial.

19.3 Delivery decision (`AdScheduler`)

Runs on every page open in the Free tier, before the page view appears.

1. If `FeatureGate.allows(.noAds)` → no ad, and the provider is never created.
2. If the inventory's `enabled` is false, or the remote kill switch `RemoteConfig.adsEnabled` is false → no ad.
3. If within `graceAppOpens` or `graceHoursAfterInstall` → no ad.
4. If the path matches `excludePaths` → no ad.
5. If the frequency-cap key (`<per>|<window start>`) already has `count` impressions → no ad.
6. If `now - lastDismissedAt < minIntervalSeconds` → no ad.
7. If today's total ≥ `maxPerDayTotal` → no ad.
8. Ask the provider for a creative. None → no ad.
9. Show it. Record the impression against the cap key, the creative's daily count and the daily total only once the creative has rendered, so a failed load does not consume the user's quota.

`localDay` uses the device calendar and time zone; the cap resets at local midnight.

19.4 Presentation

- `fullScreenCover` hosting `AdView`, shown after the page has loaded behind it so dismissal reveals a ready page.
- HTML and image creatives render in a dedicated `WKWebView` with `WKWebsiteDataStore.nonPersistent()`, no scheme handler, no bridge, JavaScript on, navigation locked to the creative's own origin. Any navigation attempt is cancelled; a tap on the creative is captured natively and routed to `clickURL`.
- A countdown pill in the top trailing corner shows the seconds until Skip; it becomes a Skip button, 44 pt minimum, never covered by the creative.
- A Remove Ads text button at the bottom opens the paywall with reason `.ad`.
- `https` clicks open `SFSafariViewController`; `webber://_paywall` opens the paywall sheet over the ad; the ad dismisses when the paywall does.
- VoiceOver: the creative is one element labelled "Advertisement", Skip is focused automatically when it appears, and Escape gesture dismisses once Skip is available.
- If the creative has not rendered within 3 seconds, dismiss silently and do not count the impression.

19.5 Measurement

Local only by default: a `UserDefaults` dictionary of `creativeId → [day: counts]` pruned to 7 days, shown in the debug menu. If `RemoteConfig.adBeaconURL` is set, send one `POST` per event with `{creativeId, event, day, appVersion, locale}` and nothing else, batched and retried with `URLSession` background configuration. No device identifiers, no IDFA, no ATT prompt. Enabling the beacon changes the privacy label (section 18.5).

19.6 `RemoteConfig`

One JSON file at a fixed `https` URL baked into the build, fetched on launch and every 6 hours, cached with ETag, bundled fallback. Everything here has a safe bundled default.

```
{
  "minVersion": "1.0",
  "adsEnabled": true,
  "adInventoryURL": "https://example.com/webber/ads.json",
  "adBeaconURL": null,
  "freeSiteLimit": 1,
  "freeFeatures": [],
  "messages": { "banner": null }
}
```

`messages.banner` shows a dismissable one-line notice in the site list, for announcements.

19.7 Debug menu (debug builds only)

Force show an ad, reset all caps, set the inventory URL to a staging file, toggle `-ForcePro`, view the local measurement table, view the resolved remote config.

20. Remote sources: the Webble protocol

A webble is a site whose source is a manifest at an `https` URL rather than a folder. Webber subscribes to the URL, pulls the files it lists into the cache, and checks for updates. Anything that can serve static files can publish a webble, so the protocol needs no backend; a builder (section 20.8) comes later and emits the same files.

20.1 Positioning and tier decision

- Webbles are free to install and read. The free site limit (section 16, row 1) applies to local folders only; `RemoteConfig.freeWebbleLimit` defaults to 10. Ads still apply on Free. If a client had to buy Pro to open their trainer's sheet, nobody would send one.
- Webbles are free to the creator in v1 too. Selling digital content for consumption inside the app brings Apple's in-app purchase rules; any charging happens between creator and client outside the app, and Webber never brokers it. Verify this reading

against the current App Store Review Guidelines before launch (section 22.2).

- The runtime is the contract, not the moat. The builder and its category templates are.

20.2 Manifest

JSON only. YAML and TOML have no parser in the iOS SDK and add nothing a generator cannot emit as JSON. The file is `webber.json`, the same schema as section 17.1 plus the keys below. A manifest with no `files` and no `bundle` is a local-folder manifest; one with either is a webble.

```
{
  "webber": 1,
  "id": "com.example.pt.sarah.week-12",
  "name": "Week 12 · Strength",
  "version": "2026.11.05",
  "entry": "index.html",
  "icon": "icon.png",
  "themeColor": "#102a43",
  "author": { "name": "Sarah K, PT", "url": "https://example.com/sarah" },
  "category": "fitness",
  "updated": "2026-11-05T08:00:00Z",
  "update": { "checkEveryHours": 12 },
  "minApp": "1.2",
  "permissions": ["storage", "haptic", "share"],
  "files": [
    { "path": "index.html", "sha256": "…", "size": 8120 },
    { "path": "style.css",  "sha256": "…", "size": 2210 },
    { "path": "app.js",     "sha256": "…", "size": 5402 },
    { "path": "icon.png",   "sha256": "…", "size": 14880 }
  ]
}
```

Key	Required	Meaning
`webber`	yes	Protocol version. The app rejects a value above the one it supports with a "please update Webber" message.
`id`	yes	Stable identity, reverse-DNS or UUID, 1–128 chars. Two manifests with the same `id` are the same webble; moving a webble to a new URL keeps its alias, storage and shortcuts.
`version`	yes	Opaque string. A change triggers a refresh; otherwise hashes decide.

Key	Required	Meaning
`files`	one of	Every file to fetch, relative to the manifest URL, with SHA-256 and size. Max 500 entries, 50 MB total.
`bundle`	one of	`{ "url": "site.zip", "sha256": "…", "size": n }` as an alternative to `files`. Zip must not contain absolute paths, `..`, or symlinks.
`author`	no	Shown on the install card and in site detail. `url` must be `https`.
`category`	no	One of `fitness`, `language`, `recipe`, `reference`, `other`. Used for list grouping and the builder.
`updated`	no	ISO 8601; shown to the user.
`update.checkEveryHours`	no	1–168; default 24. Webber may check less often.
`minApp`	no	Minimum Webber version; the install card warns if unmet.
`permissions`	no	Bridge features the webble will use: `storage`, `haptic`, `share`, `files`, `open`, `clipboard`. Shown on the install card. Calls to undeclared features reject.

All section 17.1 keys are honoured with the same tier rules, except `alias`: a webble's alias is always derived from `id` (last dot-separated segment, section 6.2 rules) and can be renamed by the user.

20.3 Install flow

1. Entry points: a universal link `https://<webber domain>/add?url=<manifest URL>`, the launch URL `webber://_add?url=<manifest URL>`, a QR code of either, or pasting the manifest URL into Add Site. The universal link is the one creators share; on a phone without Webber it lands on a web page that explains the app and links to the App Store.
2. Fetch the manifest (10 s timeout, 256 KB cap). Validate. Resolve relative file URLs against the manifest URL; every resulting URL must be `https` and must share the manifest's host unless the manifest sets `"allowCrossOrigin": true`.
3. Show the install card: icon, name, author, category, size, permissions, `minApp` warning if any. Buttons: Add, Cancel. Nothing is written before Add.

4. On Add: create a `SiteRecord` with `source: .remote(manifestURL: URL, id: String)`, then run a remote sync (20.4). On success push the page view. On failure show the error and remove the record.

20.4 Remote sync

Replaces steps 1–4 of section 7.1 for remote sources; steps 5–7 are unchanged.

1. Fetch the manifest with `If-None-Match`. 304 → nothing to do.
2. If `version` and every file hash match the stored manifest → store the new ETag, done.
3. Download changed files (or the bundle) into the tmp tree. Verify each SHA-256 and size before the swap; a mismatch aborts the whole sync and keeps the old cache.
4. Atomic swap as in 7.1. Store the new manifest beside the cache as `.webber-manifest.json`; the handler never serves dot-files.

Update checks run on page open (if older than `checkEveryHours`), on pull-to-refresh, and in a `BGAppRefreshTask` that iterates webbles due for a check. A webble whose manifest returns 404 or 410 is marked `.gone`; the cache still opens and the list shows "No longer published".

20.5 Security limits

- `https` only; no redirects to another host; no `http` fallback.
- Manifest 256 KB, file 50 MB, webble 50 MB, 500 files; counts enforced before download starts.
- Hashes verified before swap; a webble is either fully updated or untouched.
- Bridge permissions are declared in the manifest and granted on install; an updated manifest that adds a permission prompts the user before the next open.
- Webbles get the same origin isolation as any site: `webber-local://<alias>`.
- Webbles cannot read other sites, the registry, or anything outside their cache.

20.6 Universal links

Add the Associated Domains entitlement with `applinks:<webber domain>` and host `/.well-known/apple-app-site-association` with paths `/add`, `/w/*`. `/w/<token>` is a short link the backend resolves to a manifest URL; v0 is a static JSON file per token. `onOpenURL` handles both `webber://` and these `https` URLs through the same router.

20.7 Creator-side: publishing a webble by hand

A folder with `index.html` and a `webber.json` listing its files, uploaded to any static host, is a valid webble. Ship a 30-line script in the README (`make-webble.sh`) that computes the hashes and writes the `files` array, so the protocol is usable before the builder exists.

20.8 Builder (out of scope for this spec; recorded for direction)

A web app where a creator picks a category template (session sheet, cheat sheet, recipe card), fills in structured fields, and gets a hosted webble plus a `/w/<token>` link and QR code. Output is exactly the static protocol above. Later layers: versioned updates pushed to all subscribers, per-client variants of one template, creator analytics from the beacon in 19.5 if enabled. None of this changes the app; that is the point of fixing the protocol first.

21. Distribution: creators, consumers and the three revenue lines

Three parties, three products, three charges. The consumer app (Webber) is free with a one-time Pro unlock. The creator tool (Webber Creator) is a web app billed outside Apple. Webber Cloud sits between them and turns a creator's publication into a signed link a consumer can open. Nothing in this section changes the static protocol in section 20; it adds a service on top of it.

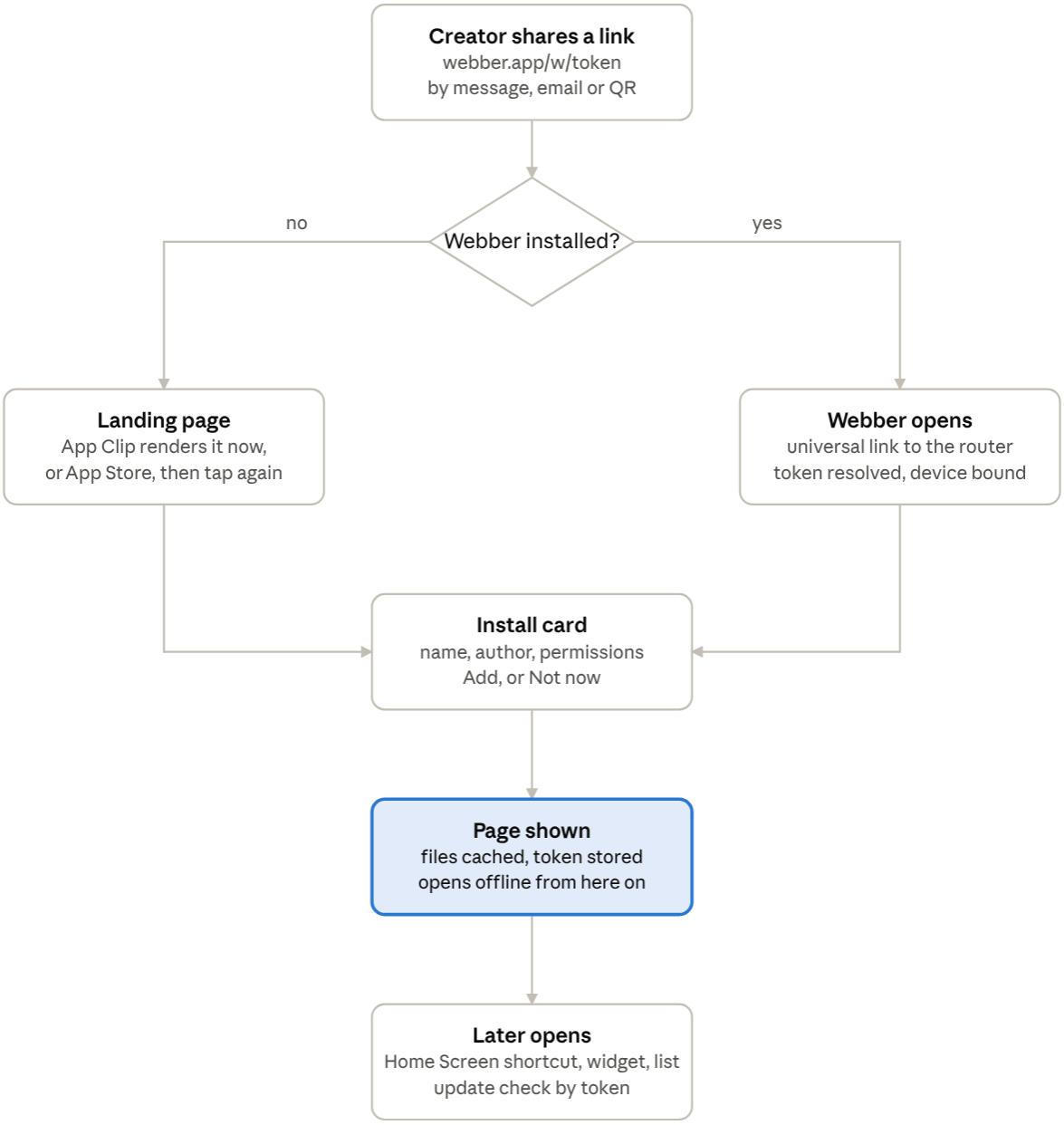
21.1 Parties and products

Party	Product	Pays for	Where billed
Consumer, Free	Webber app	Nothing. One local site, one page, webbles from any creator, interstitials per section 19	—
Consumer, Pro	Webber app	One-time unlock: unlimited local sites and pages, external folders, no ads anywhere, Pro features in section 16	App Store in-app purchase (section 18)
Creator, Free	Webber Creator	Nothing. Publishes webbles that carry Webber house ads to Free consumers and a "Made with Webber" line in the install card	—
Creator, Plus	Webber Creator	Ad-free webbles for every consumer, custom icon and theme, per-consumer links, update push, analytics; capped number of active webbles	Stripe, on the web

The creator's tier, not the consumer's, is the default lever for ads on a webble. A PT who wants clients to have a clean experience pays once; the client pays nothing and installs nothing beyond the free app. Consumer Pro removes ads too, for people who want it on their own sites.

21.2 Consumer journey

A consumer needs the link once; after that the webble lives on the phone



Consumer journey · from shared link to Home Screen

Without Webber the link lands on a page that either runs the App Clip or sends the consumer to the App Store and back; with it, the router takes the link straight to the install card. Both branches end in the same cached page.

Two facts shape this flow. iOS gives no API to put an icon on the Home Screen from a link, so the first open always lands in the app and the icon is a second step (Shortcuts or widget, section 9). And `webber://` does nothing on a phone without Webber, so the shared link is always `https://<webber domain>/w/<token>` (section 20.6).

App Clip. An App Clip bound to `/w/*` lets a consumer open the webble from the link before installing anything: iOS shows the App Clip card, the clip fetches the manifest and renders the page with the same scheme handler, and offers the full app at the end. This is the single biggest improvement to conversion available and should be prototyped in milestone 13. Constraints: the clip has a strict size limit (verify the current figure for the deployment target), no persistent storage beyond a short window, and no Shortcuts or widgets; it is a preview, not a home.

Subsequent opens. The webble is in the cache; the app opens it from the site list, a Home Screen shortcut, a widget, Spotlight or the same link tapped again. Update checks use the stored token (21.3).

21.3 Signed links, access and revocation

A creator who has sold a webble to one consumer needs that consumer, and not the internet, to have it. Consumers have no accounts; possession of the link is the credential, with device binding to limit sharing.

- `/w/<token>` is a share link. `token` is 22 URL-safe characters, random, stored by Webber Cloud against `(creator, webble, consumer label, tier, expiry, device limit)`.
- Resolution: `GET https://<webber domain>/api/v1/links/<token>` with header `X-Webber-Device: <device id>` returns `{ manifestURL, headers: { Authorization: "Bearer <short-lived JWT>" }, ads: "creator-free" | "none", expiresAt }`. The app fetches the manifest and files with that header. Static hosting still works: a creator who self-hosts omits the header and the files are public.
- Device id: a random UUID generated on first launch, stored in the keychain, never derived from hardware identifiers. First device to resolve a token claims it; the token allows `deviceLimit` devices (default 2). A creator can reset a token's devices from Webber Creator.
- Revocation: the creator revokes a link or a consumer; resolution then returns 410 and the app marks the site `.gone` (section 20.4). The cache stays readable until the consumer removes it; revocation stops updates, it does not reach into a phone and delete files. Say this plainly to creators.
- Expiry: optional. A 12-week programme can expire; the app shows "Expired" and offers to remove.
- Per-consumer variants: one webble, many links, each resolving to the same manifest or to a variant path (`/sarah/week-12/clients/<label>/webber.json`). Creator Plus only.
- Tokens are never shown to the consumer as something to type. The link is the whole experience.

21.4 Ad policy matrix

Consumer tier	Local site	Webble from Free creator	Webble from Plus creator
Free	Interstitials per section 19 (`graceOpensPerPage`, default 3, then one per page per day)	Interstitials, same policy	None
Pro	None	None	None

The app never shows an ad over a webble whose link resolved with `ads: "none"`. If resolution fails and the cache opens offline, the last known value stands.

21.5 Pricing structure

Figures are the proposal to test, not decisions; final prices are set in App Store Connect and Stripe and never hard-coded.

Charge	Proposal	Notes
Consumer Pro	£4.99 one-time, Family Sharing on	One SKU. Do not sell ad removal separately; two SKUs for one app halve the clarity of the pitch and double the review surface.
Creator Plus	Annual subscription with a cap of active webbles (e.g. 25), or pay-as-you-go per webble-month	A subscription fits the update-and-maintain use case; per-link pricing charges a PT again for every client and nothing for a year of updates. Keep per-link only as the no-commitment entry (£1 per link per 30 days) and expect it to be a minority.
Creator Free	£0	Ads to Free consumers, Webber line in the install card, 3 active webbles, no per-consumer links.

21.6 Webber Cloud, v1 scope

The smallest service that makes 21.3 true. One region, one Postgres, one worker. Elixir/Phoenix is the obvious choice given the rest of your stack; nothing here needs more than a monolith.

Endpoint	Purpose
`GET /w/<token>`	Universal-link landing: AASA-matched on iOS so the app or App Clip takes it; otherwise a web page with the webble's name, author, an App Store button and, for Plus creators, a Safari render of the entry page so the content is reachable without the app
`GET /api/v1/links/<token>`	Resolution (21.3)
`POST /api/v1/webbles` and friends	Creator CRUD, file upload, publish, revoke; authenticated creator sessions
`POST /api/v1/events`	Optional beacon (section 19.5) and webble update pings; no device identifiers beyond the random id
Stripe webhooks	Creator tier changes

Storage: webble files on object storage behind a CDN, served at `/files/<webble>/<version>/…` with the bearer check at the edge for gated webbles. Static-host creators never touch this.

Apple rules that constrain the service. Creator billing lives on the web; the iOS app contains no link, button or text that steers anyone to buy creator features, and the App Store listing describes Webber as the consumer app. Consumer Pro is only ever sold through StoreKit. Verify both against the current guidelines for each storefront before launch (section 22.2).

21.7 What this changes in earlier sections

- Section 16, row 15: webbles stay free to consume; the row's note now reads with 21.4.
- Section 18: unchanged SKU; the paywall gains one line, "Webbles sent to you stay free".
- Section 19.2: add `graceOpensPerPage` (default 3) beside `graceAppOpens`; the scheduler checks both. Add `creatorTier` to `AdContext` so step 2 of 19.3 can skip ads on `ads: "none"` webbles.
- Section 20.3: the install card shows "Made with Webber" for Free-creator webbles and the creator's name without it for Plus.
- Section 20.4: remote sync sends the bearer header when the link resolution supplied one, and re-resolves the token when the JWT expires.
- Section 20.6: `/w/<token>` is now served by Webber Cloud, not a static file.

- Milestone 13 gains: link resolution client, device id, `.gone` and "Expired" states, App Clip target.

22. Open questions and decisions to validate on device

These are assumptions in this spec that I cannot verify without running code. Each one can change the design, so test them in milestone 3 or earlier.

1. **Custom scheme as a secure context.** WebKit may treat `webber-local://` as insecure and refuse `crypto.subtle`, `navigator.clipboard`, or module workers. Test with a page that exercises each API the user's existing pages use. If it fails, the fallback is to name the scheme with WebKit's secure-scheme heuristics in mind, or to load via `loadFileURL(_:allowingReadAccessTo:)` and accept the `fetch` limits.
2. **`UTType` coverage for `.mjs` and `.wasm`** on iOS 17. The override table in 8.4 assumes gaps; confirm which exist.
3. **`NSMetadataQuery` latency** for new folders in the container. If it takes longer than about 30 s to notice a new folder, add a pull-to-refresh that enumerates the Documents directory directly.
4. **Dropbox through File Provider offline.** Bookmark resolution and enumeration may fail when Dropbox has not materialised the folder. Confirm whether `startDownloadingUbiquitousItem` applies to non-iCloud providers (expected: no) and whether reading a placeholder triggers a download or throws.
5. **Web Clip profiles with a custom scheme.** A configuration profile Web Clip might accept `webber://` and give a real icon without Shortcuts. Unverified; worth 20 minutes with Apple Configurator.
6. **Caches purge frequency.** If iOS purges the cache often, move it to `Application Support` with `isExcludedFromBackup = true`. Decide after a week of use.
7. **Alias in the URL versus a UUID.** Aliases are readable and typeable but break on folder rename. A UUID in the URL survives renames but is unreadable. This spec picks aliases plus the Moved grace period. Revisit if renames turn out to be common.
8. **Widget versus Shortcuts as the primary Home Screen path.** If the Shortcuts flow feels too heavy in practice, promote the widget from optional to milestone 8 and make the intent secondary.

22.2 Added by the tiering, ad and webble design

1. **Interstitial on open and App Review.** Reviewers have rejected apps whose first interaction is an ad. The grace rules in 19.2 exist for this; confirm the reviewer's device never sees an ad by making `graceAppOpens` at least 3 in the shipped inventory.

2. **Second-HTML-document gate.** Enforced in the navigation policy, so a Free site built as a hash-routed single page is unlimited. Decide whether that is a loophole or the intended shape of the free tier. This spec says intended.
3. **House ads only at launch.** Confirm whether AdMob or similar is ever worth the SDK and the privacy label. Until a Pro conversion rate exists there is nothing to compare it against.
4. **KVS 1 MB limit** shared across all sites' bridge storage and the registry sync. If a user has many sites with heavy `webber.storage` use, move bridge storage to CloudKit private database.
5. **`BGAppRefreshTask` scheduling** is best-effort; fetch-from-URL on a schedule may run hours late or not at all. Document it as "when iOS allows".
6. **Control Center controls** require iOS 18; the deployment target stays 17 and the control is conditionally compiled.

22.3 Added by the Webble protocol

1. **App Store rules for third-party web content.** Confirm against the current guidelines that an app which installs and renders creator-published HTML, with a limited native bridge, is acceptable, and that keeping all payment off-platform avoids the in-app purchase requirement. Do this before any creator sees the protocol.
2. **Universal link domain.** Needs a domain, an AASA file and an App Store listing to fall through to. Nothing in the protocol works for a creator's clients until this exists.
3. **Protocol freeze.** Once a creator publishes, `webber: 1` is frozen. Additive keys only; a breaking change is `webber: 2` with the app supporting both.
4. **Chicken and egg.** A PT's client must install Webber to open the sheet. Measure how many links sent result in an install before building the builder; if the number is poor, the fallback web view at `/w/<token>` has to render the webble in Safari too, which makes Webber optional and changes the pitch.

22.4 Added by the distribution model

1. **App Clip size and capability limits** for the deployment target, and whether the scheme handler and a cached manifest fit inside them. If not, the landing page's Safari render carries the no-app case alone.
2. **Device binding versus households.** A limit of 2 devices breaks a consumer with an iPhone and an iPad plus a family member. Decide whether the limit is per link or per consumer label, and whether a reset is self-service.
3. **Creator pricing.** Subscription with a cap against per-link: run both with the first ten creators and keep the one they renew.

4. **Free-creator ads on paid content.** A consumer who paid a creator and then sees a Webber ad will blame the creator. Consider making any webble the creator marks as sold ad-free regardless of tier, and charging the creator for that flag instead.
5. **Store rules by region** for a consumer app whose sibling web product is billed outside Apple; the constraints in 21.6 are the conservative reading and may be looser in some storefronts.

22.5 Added by the Home Screen decision

1. **`shortcuts://import-shortcut` on current iOS.** Confirm the scheme still imports a signed `.shortcut` from an `https` URL on iOS 17 and 18, and what the import sheet looks like. If it has been removed, the primary route collapses to the written steps and the widget becomes primary.
2. **Server-side signing.** `shortcuts sign --mode anyone` runs on macOS only; Webber Cloud needs a Mac (a Mac mini or a macOS CI runner) in the publish path. Confirm the signed file stays valid when the embedded URL changes per link, or whether one generic signed shortcut with a text parameter is acceptable.
3. **Widget tier.** Widgets moved to Free because they are a Home Screen path for webble consumers; confirm this does not hollow out the Pro pitch in section 16.